

模型代码 PR : [transformers/pull/43830](https://github.com/huggingface/transformers/pull/43830)

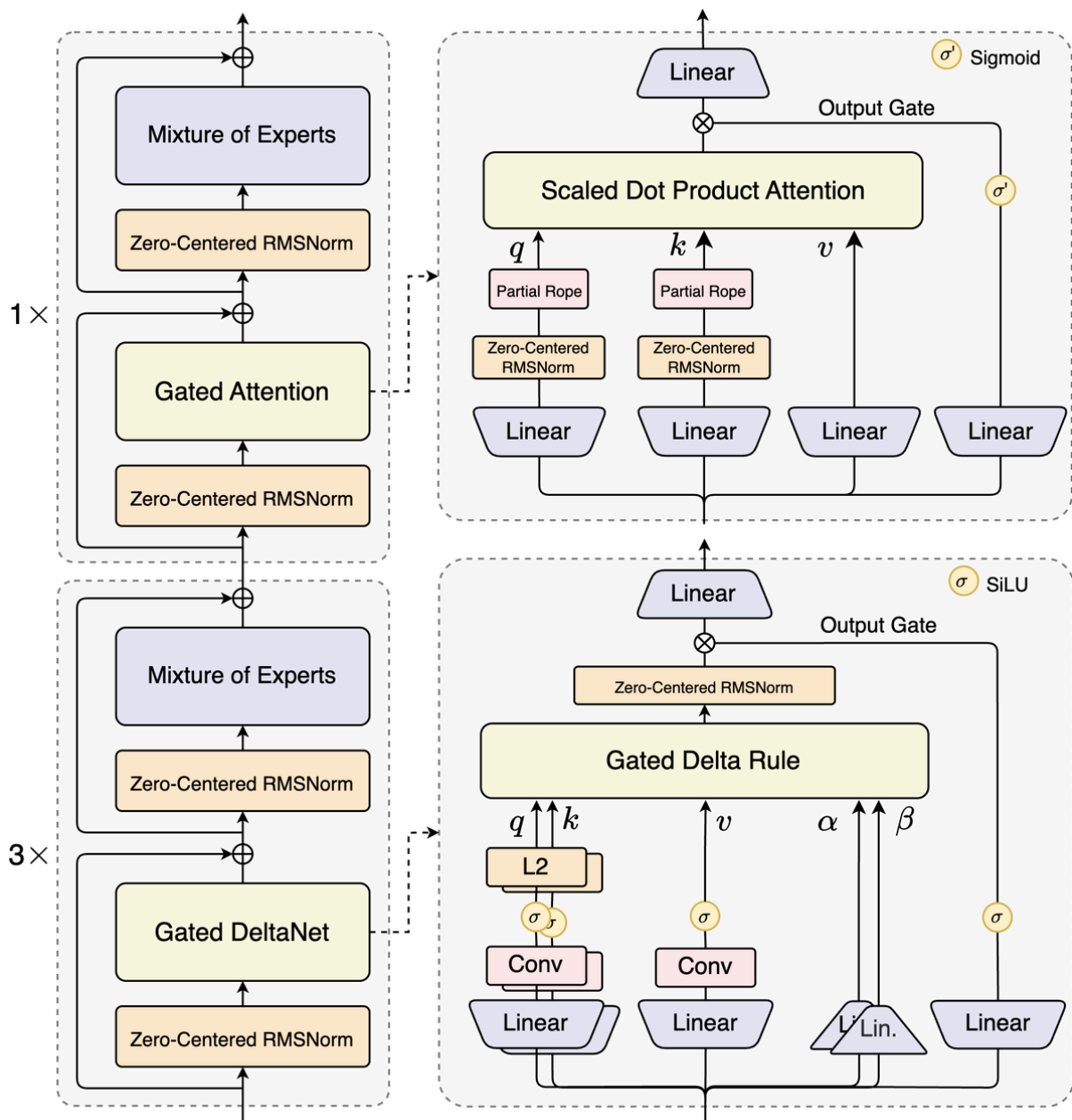
本次开源的代码有 Qwen-3.5 和 Qwen-3.5-MOE 两类多模态大模型，本质上来说：

- **Qwen-3.5 就是多模态版的 Qwen3 Next**，即把 ViT + PatchMerger 接上 [Qwen3 Next](#)。自然地，模型也支持 Text-Only 的输出。
- **Qwen-3.5-MOE 在 Qwen-3.5 基础上将 LLM Backbone 的 FFN 替换为 Qwen3 风格的 MoE** (Top-K Router + Multi-Expert + Shared Expert)，其中 Shared Expert 是 Qwen3 没有的

1. Qwen-3.5

Qwen-3.5 是一个 Dense 多模态大模型（当然也支持Text-Only模式）：

- 多模态架构上相比[Qwen3-VL](#)去除了复杂的 [DeepStack](#) 结构，回归了[Qwen2-VL](#)/[Qwen2.5-VL](#)的 ViT + PatchMerger + LLM 的三段式结构。
- LLM 架构上 Qwen-3.5 参考 [Qwen3 Next](#) 使用支持交错 Gated/Linear Attention 架构（每4层：3层 GatedDeltaNet + 1层 Gated Attention）
- 整体上看 Qwen-3.5 就是多模态版的 [Qwen3 Next](#)。



1.1 Vision Encoder

ViT 架构回归了 [Qwen2-VL](#) / [Qwen2.5-VL](#) 的 [NaViT](#) 架构，不再使用 [Qwen3-VL](#) 的 [DeepStack](#)：

```
class Qwen3_5VisionModel(Qwen3_5PreTrainedModel):
    ...
    def __init__(self, config, *inputs, **kwargs) -> None:
        super().__init__(config, *inputs, **kwargs)
        self.spatial_merge_size = config.spatial_merge_size
        self.patch_size = config.patch_size
        self.spatial_merge_unit = self.spatial_merge_size * self.spatial_merge_size
```

```

        self.patch_embed = Qwen3_5VisionPatchEmbed(
            config=config,
        )
        self.pos_embed = nn.Embedding(config.num_position_embeddings,
config.hidden_size)
        self.num_grid_per_side = int(config.num_position_embeddings**
0.5)
        head_dim = config.hidden_size // config.num_heads
        self.rotary_pos_emb = Qwen3_5VisionRotaryEmbedding(head_dim
// 2)
        self.blocks = nn.ModuleList([Qwen3_5VisionBlock(config) for _
in range(config.depth)])
        self.merger = Qwen3_5VisionPatchMerger(
            config=config,
            use_postshuffle_norm=False,
        )

```

1.2 Projector

Projector 沿用 [Qwen2-VL](#) / [Qwen2.5-VL](#) / [Qwen3-VL](#) 的 PatchMerger 结构：

```

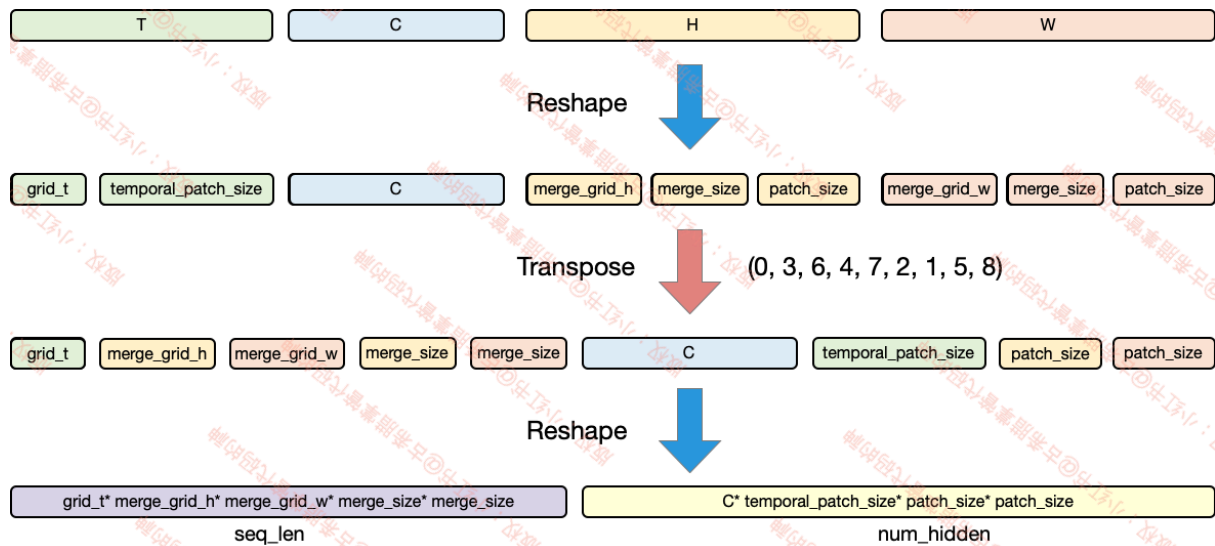
class Qwen3_5VisionPatchMerger(nn.Module):
    def __init__(self, config: Qwen3_5VisionConfig, use_postshuffle_n
orm=False) -> None:
        super().init ()
        self.hidden_size = config.hidden_size * (config.spatial_merge
_size**2)
        self.use_postshuffle_norm = use_postshuffle_norm
        self.norm = nn.LayerNorm(self.hidden_size if use_postshuffle_
norm else config.hidden_size, eps=1e-6)
        self.linear_fc1 = nn.Linear(self.hidden_size, self.hidden_siz
e)
        self.act_fn = nn.GELU()
        self.linear_fc2 = nn.Linear(self.hidden_size, config.out_hidd
en_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.norm(x.view(-1, self.hidden_size) if self.use_postsh
uffle_norm else x).view(-1, self.hidden_size)

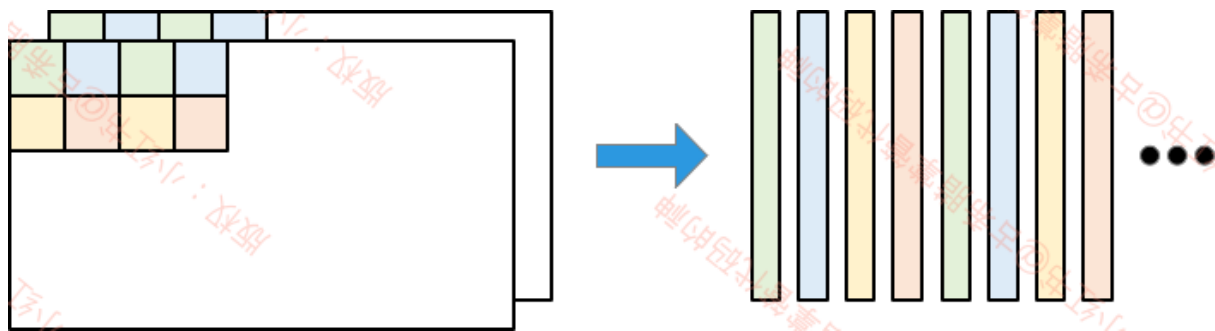
```

```
x = self.linear_fc2(self.act_fn(self.linear_fc1(x)))
return x
```

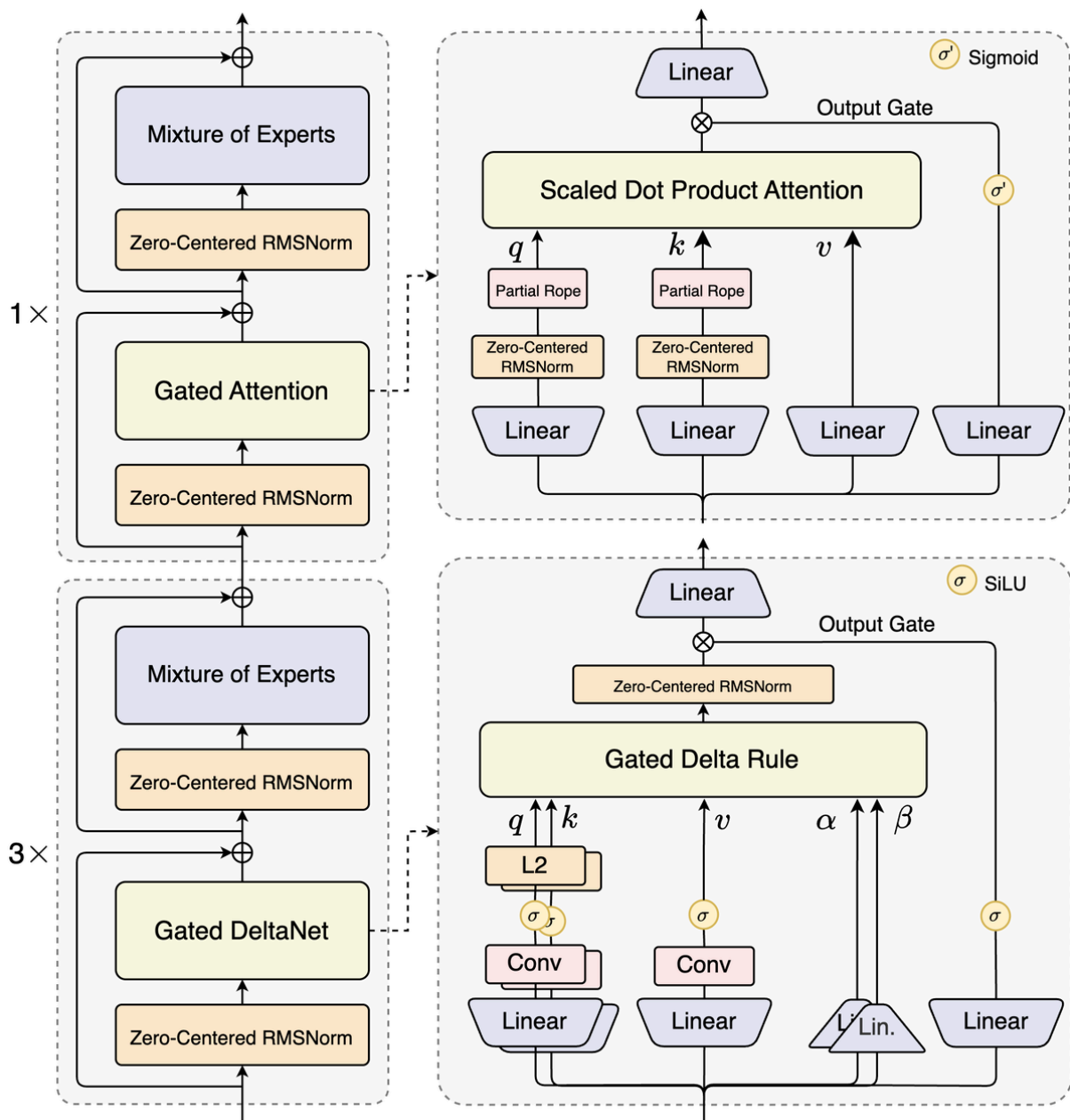
在ViT内部（PatchMerger之前）维度经过切分重组，保证2x2的patch相邻：具体合并方式是通过reshape分割H，W维度之后，将两者的patch size放在num_hidden维度的末尾。而两个merge_size放在seq_len的末尾，这意味着遍历reshape的时候，是2*2的patch在seq_len维度相邻（详细可以参考[PatchMerger是如何合并2x2的Patch的？](#)）：



即下图的排列方式：



1.3 Qwen-3.5 LLM Backbone



LLM 最重要的特点是交错 Gated/Linear Attention 架构（本质上参考的 Qwen），默认是每4层Transformer Block中有3层是 GatedDeltaNet 有1层是 Gated Attention（**比例为3:1**），参考以下代码：

```
self.layer_types = layer_types
if self.layer_types is None:
    interval_pattern = kwargs.get("full_attention_interval", 4)
    self.layer_types = [
        "linear_attention" if bool((i + 1) % interval_pattern) else
        "full_attention"
        for i in range(self.num_hidden_layers)
    ]
```

```

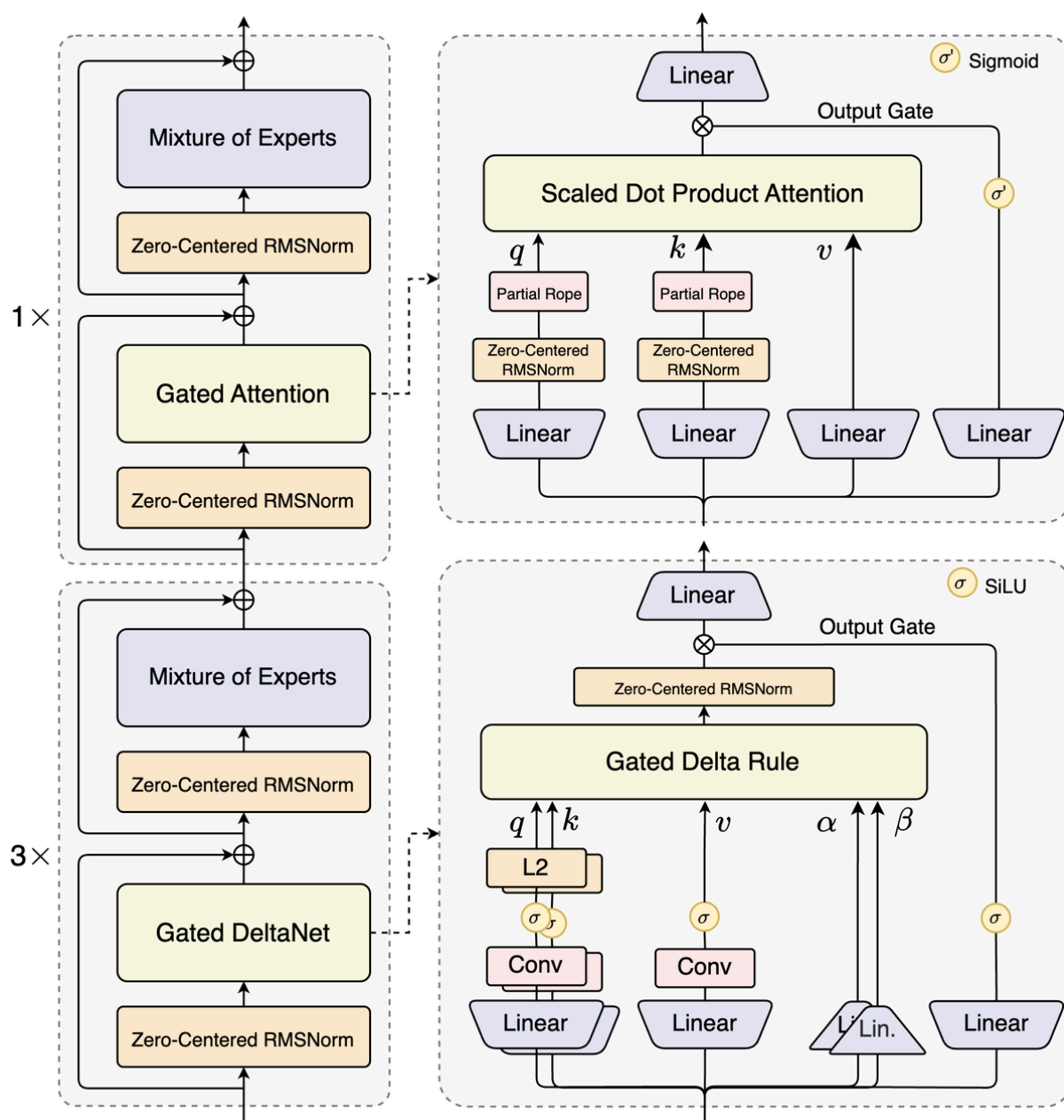
]
layer_type_validation(self.layer_types, self.num_hidden_layers)

```

1.3.1 Gated Attention

首先来看Gated Attention 的两个关键特点：

- 使用了QK Norm，并且所使用的 RMSNorm 也是 Qwen3-Next 风格的 **Zero-Centered LayerNorm**。
- `q_proj` 输出维度是 *2，并显式拆出 `gate`，**输出被 learned gate 逐 token/逐 head_dim 调制。**



```

class Qwen3_5Attention(nn.Module):
    def __init__(self, config: Qwen3_5Config, layer_idx: int):
        super().__init__()
        self.config = config
        self.layer_idx = layer_idx
        self.head_dim = getattr(config, "head_dim", config.hidden_size // config.num_attention_heads)
        self.num_key_value_groups = config.num_attention_heads // config.num_key_value_heads
        self.scaling = self.head_dim**-0.5
        self.attention_dropout = config.attention_dropout
        self.is_causal = True
        self.q_proj = nn.Linear(
            config.hidden_size, config.num_attention_heads * self.head_dim * 2, bias=config.attention_bias
        )
        self.k_proj = nn.Linear(
            config.hidden_size, config.num_key_value_heads * self.head_dim, bias=config.attention_bias
        )
        self.v_proj = nn.Linear(
            config.hidden_size, config.num_key_value_heads * self.head_dim, bias=config.attention_bias
        )
        self.o_proj = nn.Linear(
            config.num_attention_heads * self.head_dim, config.hidden_size, bias=config.attention_bias
        )
        self.q_norm = Qwen3_5RMSNorm(self.head_dim, eps=config.rms_norm_eps) # unlike olmo, only on the head dim!
        self.k_norm = Qwen3_5RMSNorm(self.head_dim, eps=config.rms_norm_eps) # thus post q_norm does not need reshape

    def forward(
        self,
        hidden_states: torch.Tensor,
        ...
    ) -> tuple[torch.Tensor, torch.Tensor | None]:
        input_shape = hidden_states.shape[:-1]
        hidden_shape = (*input_shape, -1, self.head_dim)

```

```

        query_states, gate = torch.chunk(
            self.q_proj(hidden_states).view(*input_shape, -1, self.head dim * 2), 2, dim=-1
        )
        gate = gate.reshape(*input_shape, -1)
        query_states = self.q_norm(query_states.view(hidden_shape)).transpose(1, 2)
        key_states = self.k_norm(self.k_proj(hidden_states).view(hidden_shape)).transpose(1, 2)
        value_states = self.v_proj(hidden_states).view(hidden_shape).transpose(1, 2)
        ...
        attn_output, attn_weights = attention_interface(
            ...
        )
        attn_output = attn_output.reshape(*input_shape, -1).contiguous()

        attn_output = attn_output * torch.sigmoid(gate)
        attn_output = self.o_proj(attn_output)
        return attn_output, attn_weights

```

另外，Zero-Centered LayerNorm 代码实现如下，之所以叫Zero-Centered，是因为 `self.weight = nn.Parameter(torch.zeros(dim))` 是进行0初始化的，这样一来，初始时就类似恒等映射（`scale=1`），即标准化，不会因为 RMSNorm 未减均值造成训练不稳定：

```

class Qwen3_5RMSNorm(nn.Module):
    def __init__(self, hidden_size, eps: float = 1e-6) -> None:
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.zeros(hidden_size))

    def _norm(self, x):
        return x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)

    def forward(self, hidden_states: torch.Tensor) -> torch.Tensor:
        output = self._norm(hidden_states.float())
        # Llama does x.to(float16) * w whilst Qwen3_5 is (x * w).to(float16)
        # See https://github.com/huggingface/transformers/pull/29402

```



```

        output = output * (1.0 + self.weight.float())
        return output.type_as(hidden_states)

    def extra_repr(self):
        return f"{tuple(self.weight.shape)}, eps={self.eps}"

```

对比标准RMSNorm实现：

```

import torch
import torch.nn as nn

class RMSNorm(nn.Module):
    """Root-Mean-Square Normalization"""
    def __init__(self, features, eps: float = 1e-6):
        super(RMSNorm, self).__init__()
        self.gamma = nn.Parameter(torch.ones(features))
        self.eps = eps

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        rms = x.pow(2).mean(dim=-1, keepdim=True).add(self.eps).sqrt()
        return self.gamma * x / rms

# Example usage
features = 768
rms_norm = RMSNorm(features)

# Dummy input tensor (batch_size, seq_length, features)
x = torch.randn(10, 20, features)
normalized_x = rms_norm(x)
print(normalized_x)

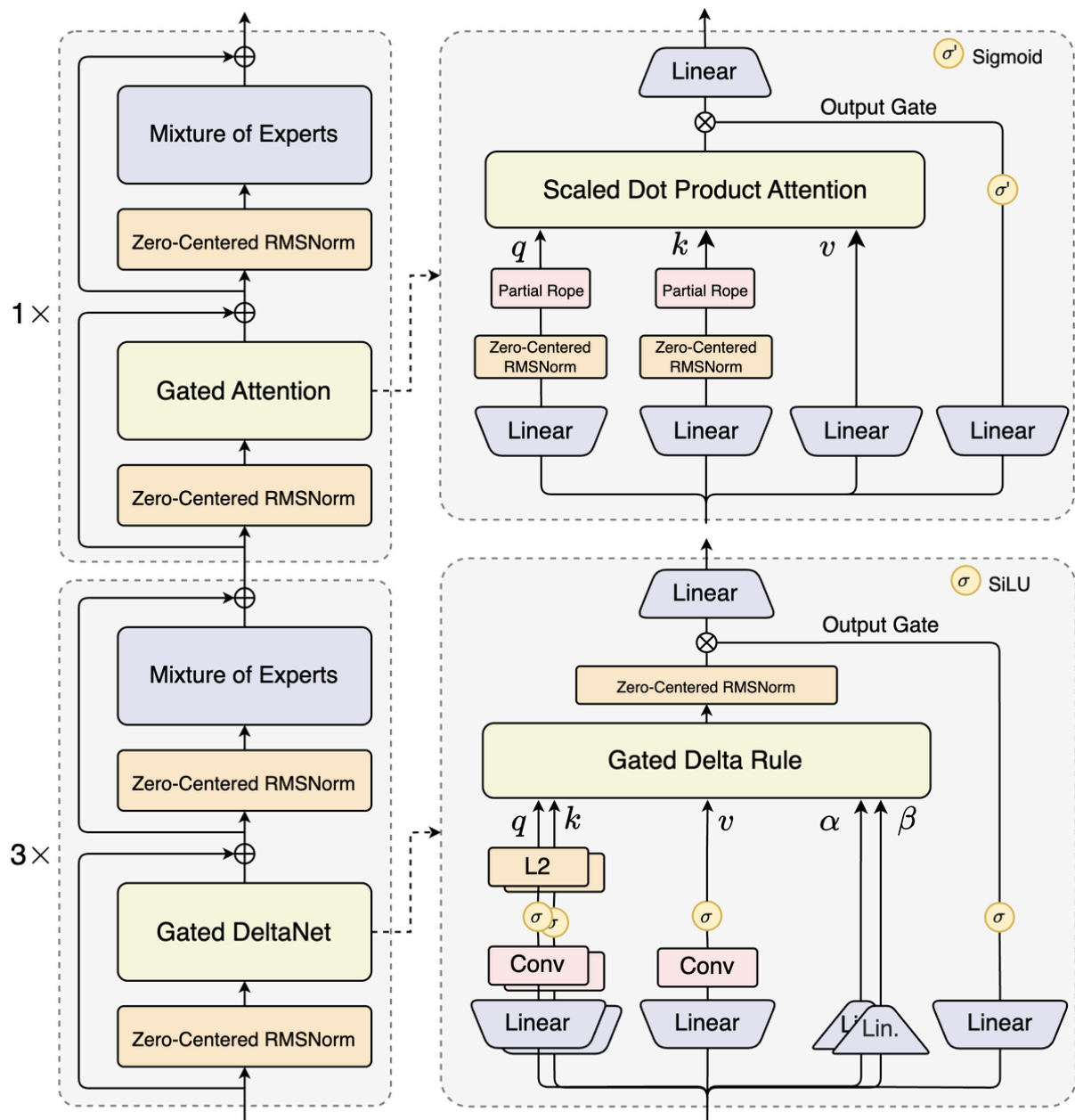
```

1.3.2 GatedDeltaNet (Linear Attention)

Gated DeltaNet 用门控 (gate) 与可学习指数衰减 (decay) 把历史信息写入一个递推状态，再用查询读出。这样把原本 $O(T^2)$ 的 Softmax Attention 换成 $O(T \cdot d)$ 的递推形式，核心递推（每个 value-head，每个时刻 t ）可写成：

$$s_t = \lambda_t \odot s_{t-1} + \beta_t \odot (\Phi(k_t) \otimes v_t), \\ y_t = \Phi(q_t) \odot s_t$$

其中 $0 < \lambda_t < 1$ 是遗忘因子（由 $g_t \leq 0$ 决定， $\lambda_t = e^{g_t}$ ）， $\beta_t \in (0, 1)$ 是输入门， $\Phi(\cdot)$ 是对 q, k 的特征映射／归一化。



核心 gated_delta_rule 代码：

```
def torch_recurrent_gated_delta_rule(
    query, key, value, g, beta, initial_state, output_final_state, use_qk_l2norm_in_kernel=False
):
    initial_dtype = query.dtype
    if use_qk_l2norm_in_kernel:
        query = l2norm(query, dim=-1, eps=1e-6)
```

```

        key = l2norm(key, dim=-1, eps=1e-6)
    query, key, value, beta, g = [
        x.transpose(1, 2).contiguous().to(torch.float32) for x in (qu
    ery, key, value, beta, g)
    ]

    batch_size, sequence_length, num_heads, k_head_dim = key.shape
    v_head_dim = value.shape[-1]
    scale = 1 / (query.shape[-1] ** 0.5)
    query = query * scale

    core_attn_out = torch.zeros(batch_size, sequence_length, num_head
s, v_head_dim).to(value)
    last_recurrent_state = (
        torch.zeros(batch_size, sequence_length, k_head_dim, v_head_d
im).to(value)
        if initial_state is None
        else initial_state.to(value)
    )

    for i in range(num_heads):
        q_t = query[:, :, i]
        k_t = key[:, :, i]
        v_t = value[:, :, i]
        g_t = g[:, :, i].exp().unsqueeze(-1).unsqueeze(-1)
        beta_t = beta[:, :, i].unsqueeze(-1)

        last_recurrent_state = last_recurrent_state * q_t
        kv_mem = (last_recurrent_state * k_t.unsqueeze(-1)).sum(dim=-
2)

        delta = (v_t - kv_mem) * beta_t
        last_recurrent_state = last_recurrent_state + k_t.unsqueeze(-
1) * delta.unsqueeze(-2)
        core_attn_out[:, :, i] = (last_recurrent_state * q_t.unsqueez
e(-1)).sum(dim=-2)

    if not output_final_state:
        last_recurrent_state = None
    core_attn_out = core_attn_out.transpose(1, 2).contiguous().to(ini

```

```

    tial_dtype)
    return core_attn_out, last_recurrent_state

```

Qwen3_5GatedDeltaNet 代码：

```

class Qwen3_5GatedDeltaNet(nn.Module):
    def __init__(self, config: Qwen3_5Config, layer_idx: int):
        super().__init__()
        self.hidden_size = config.hidden_size
        self.num_v_heads = config.linear_num_value_heads
        self.num_k_heads = config.linear_num_key_heads
        self.head_k_dim = config.linear_key_head_dim
        self.head_v_dim = config.linear_value_head_dim
        self.key_dim = self.head_k_dim * self.num_k_heads
        self.value_dim = self.head_v_dim * self.num_v_heads

        self.conv_kernel_size = config.linear_conv_kernel_dim
        self.layer_idx = layer_idx
        self.activation = config.hidden_act
        self.act = ACT2FN[config.hidden_act]
        self.layer_norm_epsilon = config.rms_norm_eps

        # QKV
        self.conv_dim = self.key_dim * 2 + self.value_dim
        self.conv1d = nn.Conv1d(
            in_channels=self.conv_dim,
            out_channels=self.conv_dim,
            bias=False,
            kernel_size=self.conv_kernel_size,
            groups=self.conv_dim,
            padding=self.conv_kernel_size - 1,
        )

        # time step projection (discretization)
        # instantiate once and copy inv_dt in init_weights of PretrainedModel
        self.dt_bias = nn.Parameter(torch.ones(self.num_v_heads))

        A = torch.empty(self.num_v_heads).uniform_(0, 16)
        self.A_log = nn.Parameter(torch.log(A))

```

```

        self.norm = (
            Qwen3_5RMSNormGated(self.head_v_dim, eps=self.layer_norm_epsilon)

            if FusedRMSNormGated is None
            else FusedRMSNormGated(
                self.head_v_dim,
                eps=self.layer_norm_epsilon,
                activation=self.activation,
                device=torch.cuda.current_device(),
                dtype=config.dtype if config.dtype is not None else torch.get_default_dtype(),
            )
        )

        self.out_proj = nn.Linear(self.value_dim, self.hidden_size, bias=False)

        self.causal_conv1d_fn = causal_conv1d_fn
        self.causal_conv1d_update = causal_conv1d_update or torch_causal_conv1d_update
        self.chunk_gated_delta_rule = chunk_gated_delta_rule or torch_chunk_gated_delta_rule
        self.recurrent_gated_delta_rule = fused_recurrent_gated_delta_rule or torch_recurrent_gated_delta_rule

        if not is_fast_path_available:
            logger.warning once(
                "The fast path is not available because one of the required library is not installed. Falling back to "
                "torch implementation. To install follow https://github.com/fla-org/flash-linear-attention#installation and "
                "https://github.com/Dao-AILab/causal-conv1d"
            )

        self.in_proj_qkv = nn.Linear(self.hidden_size, self.key_dim * 2 + self.value_dim, bias=False)
        self.in_proj_z = nn.Linear(self.hidden_size, self.value_dim, bias=False)
        self.in_proj_b = nn.Linear(self.hidden_size, self.num_v_head

```

```

s, bias=False)
        self.in_proj_a = nn.Linear(self.hidden_size, self.num_v_heads, bias=False)

    def forward(
        self,
        hidden_states: torch.Tensor,
        cache_params: Qwen3 5DynamicCache | None = None,
        cache_position: torch.LongTensor | None = None,
        attention_mask: torch.Tensor | None = None,
    ):
        hidden_states = apply_mask_to_padding_states(hidden_states, attention_mask)

        # Set up dimensions for reshapes later
        batch_size, seq_len, _ = hidden_states.shape

        use_precomputed_states = (
            cache_params is not None
            and cache_params.has_previous_state
            and seq_len == 1
            and cache_position is not None
        )

        # getting projected states from cache if it exists
        if cache_params is not None:
            conv_state = cache_params.conv_states[self.layer_idx]
            recurrent_state = cache_params.recurrent_states[self.layer_idx]

        mixed_qkv = self.in_proj_qkv(hidden_states)
        mixed_qkv = mixed_qkv.transpose(1, 2)

        z = self.in_proj_z(hidden_states)
        z = z.reshape(batch_size, seq_len, -1, self.head_v_dim)

        b = self.in_proj_b(hidden_states)
        a = self.in_proj_a(hidden_states)

        if use_precomputed_states:

```

```

        # 2. Convolution sequence transformation
        # NOTE: the conv state is updated in `causal_conv1d_update`
e`
        mixed_qkv = self.causal_conv1d_update(
            mixed_qkv,
            conv_state,
            self.conv1d.weight.squeeze(1),
            self.conv1d.bias,
            self.activation,
        )
    else:
        if cache_params is not None:
            conv_state = F.pad(mixed_qkv, (self.conv_kernel_size
- mixed_qkv.shape[-1], 0))
            cache_params.conv_states[self.layer_idx] = conv_state
        if self.causal_conv1d_fn is not None:
            mixed_qkv = self.causal_conv1d_fn(
                x=mixed_qkv,
                weight=self.conv1d.weight.squeeze(1),
                bias=self.conv1d.bias,
                activation=self.activation,
                seq_idx=None,
            )
        else:
            mixed_qkv = F.silu(self.conv1d(mixed_qkv)[: , : , :seq_
len]))

    mixed_qkv = mixed_qkv.transpose(1, 2)
    query, key, value = torch.split(
        mixed_qkv,
        [
            self.key_dim,
            self.key_dim,
            self.value_dim,
        ],
        dim=-1,
    )

    query = query.reshape(batch_size, seq_len, -1, self.head_k_di
m)

```

```

        key = key.reshape(batch_size, seq_len, -1, self.head_k_dim)
        value = value.reshape(batch_size, seq_len, -1, self.head_v_dim)

        beta = b.sigmoid()
        # If the model is loaded in fp16, without the .float() here,
        # A might be -inf
        g = -self.A_log.float().exp() * F.softplus(a.float() + self.d
        t_bias)

        if self.num_v_heads // self.num_k_heads > 1:
            query = query.repeat_interleave(self.num_v_heads // self.
            num_k_heads, dim=2)
            key = key.repeat_interleave(self.num_v_heads // self.num_
            k_heads, dim=2)

        if not use_precomputed_states:
            core_attn_out, last_recurrent_state = self.chunk_gated_de
            lta_rule(
                query,
                key,
                value,
                g=g,
                beta=beta,
                initial_state=None,
                output_final_state=cache_params is not None,
                use_qk_l2norm_in_kernel=True,
            )

        else:
            core_attn_out, last_recurrent_state = self.recurrent_gate
            d_delta_rule(
                query,
                key,
                value,
                g=g,
                beta=beta,
                initial_state=recurrent_state,
                output_final_state=cache_params is not None,
                use_qk_l2norm_in_kernel=True,
            )

```



```

        # Update cache
        if cache_params is not None:
            cache_params.recurrent_states[self.layer_idx] = last_recurrent_state

        # reshape input data into 2D tensor
        core_attn_out = core_attn_out.reshape(-1, self.head_v_dim)
        z = z.reshape(-1, self.head_v_dim)
        core_attn_out = self.norm(core_attn_out, z)
        core_attn_out = core_attn_out.reshape(batch_size, seq_len, -1)

        output = self.out_proj(core_attn_out)
        return output

```

2. Qwen-3.5-MoE

关键差异：MoE 版本把 LLM Backbone 的 FFN 从 Dense MLP 换成了 Sparse MoE (Top-K Router + Expert + Shared expert)，并且额外提供 router_logits 记录与 load-balancing aux loss 的支持，其余部分一致。

区别在于替换了 FFN：`self.mlp = Qwen3_5MoeSparseMoeBlock(config)`

每个 `Qwen3_5MoeSparseMoeBlock` 内部由 3 部分组成：

Router (Top-K)： `Qwen3_5MoeTopKRouter`

- `router_logits = softmax(W_router x)`
- `topk` 选 `num_experts_per_tok` 个专家
- `router_top_value` 归一化作为 token 的 expert 权重

```

class Qwen3_5MoeTopKRouter(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.top_k = config.num_experts_per_tok
        self.num_experts = config.num_experts
        self.hidden_dim = config.hidden_size
        self.weight = nn.Parameter(torch.zeros(self.num_experts, self.hidden_dim))

```

```

def forward(self, hidden_states):
    hidden_states = hidden_states.reshape(-1, self.hidden_dim)
    router_logits = F.linear(hidden_states, self.weight) # (seq_
len, num_experts)
    router_logits = torch.nn.functional.softmax(router_logits, dt
ype=torch.float, dim=-1)
    router_top_value, router_indices = torch.topk(router_logits,
self.top_k, dim=-1) # (seq len, top k)
    router_top_value /= router_top_value.sum(dim=-1, keepdim=True)

    router_top_value = router_top_value.to(router_logits.dtype)
    router_scores = router_top_value
    return router_logits, router_scores, router_indices

```

Experts (本质上是多个 FFN 权重) : Qwen3_5MoeExperts

- 权重以 3D 张量存储：(num_experts, ...)，实际执行把被路由到该 expert 的 token gather 出来跑对应 expert 的 MLP，再 scatter/index_add 回去。

```

class Qwen3_5MoeExperts(nn.Module):
    """Collection of expert weights stored as 3D tensors."""

    def __init__(self, config):
        super().__init__()
        self.num_experts = config.num_experts
        self.hidden_dim = config.hidden_size
        self.intermediate_dim = config.moe_intermediate_size
        self.gate_up_proj = nn.Parameter(torch.empty(self.num_experts, 2 * self.intermediate_dim, self.hidden_dim))
        self.down_proj = nn.Parameter(torch.empty(self.num_experts, self.hidden_dim, self.intermediate_dim))
        self.act_fn = ACT2FN[config.hidden_act]

    def forward(
        self,
        hidden_states: torch.Tensor,
        top_k_index: torch.Tensor,
        top_k_weights: torch.Tensor,
    ) -> torch.Tensor:
        final_hidden_states = torch.zeros_like(hidden_states)
        with torch.no_grad():

```

```

        expert_mask = torch.nn.functional.one_hot(top_k_index, num_classes=self.num_experts)
        expert_mask = expert_mask.permute(2, 1, 0)
        expert_hit = torch.greater(expert_mask.sum(dim=(-1, -2)), 0).nonzero()

        for expert_idx in expert_hit:
            expert_idx = expert_idx[0]
            if expert_idx == self.num_experts:
                continue
            top_k_pos, token_idx = torch.where(expert_mask[expert_idx])

            current_state = hidden_states[token_idx]
            gate, up = nn.functional.linear(current_state, self.gate_up_proj[expert_idx]).chunk(2, dim=-1)
            current_hidden_states = self.act_fn(gate) * up
            current_hidden_states = nn.functional.linear(current_hidden_states, self.down_proj[expert_idx])
            current_hidden_states = current_hidden_states * top_k_weights[token_idx, top_k_pos, None]
            final_hidden_states.index_add_(0, token_idx, current_hidden_states.to(final_hidden_states.dtype))

        return final_hidden_states

```

Shared Expert (共享稠密专家，本质上是必须经过的FFN，但是要做gating)

- `shared_expert = Qwen3_5MoeMLP(..., shared_expert_intermediate_size)`
- `shared_expert_gate = Linear(hidden, 1)`，用 `sigmoid` 做 gating
- 最终：`expert_output += sigmoid(shared_gate)*shared_expert_output`

```

class Qwen3_5MoeSparseMoeBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.gate = Qwen3_5MoeTopKRouter(config)
        self.experts = Qwen3_5MoeExperts(config)
        self.shared_expert = Qwen3_5MoeMLP(config, intermediate_size=config.shared_expert_intermediate_size)
        self.shared_expert_gate = torch.nn.Linear(config.hidden_size,

```

```
1, bias=False)
```

```
def forward(self, hidden_states: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:  
    batch_size, sequence_length, hidden_dim = hidden_states.shape  
    hidden_states_reshaped = hidden_states.view(-1, hidden_dim)  
    shared_expert_output = self.shared_expert(hidden_states_reshaped)  
  
    _, routing_weights, selected_experts = self.gate(hidden_states_reshaped)  
    expert_output = self.experts(hidden_states_reshaped, selected_experts, routing_weights)  
  
    shared_expert_output = F.sigmoid(self.shared_expert_gate(hidden_states_reshaped)) * shared_expert_output  
  
    expert_output += shared_expert_output  
    expert_output = expert_output.reshape(batch_size, sequence_length, hidden_dim)  
    return expert_output
```